# Reference: Working Out Boolean Expressions

Keep this beside you when you solve a True or False question. Every boolean expression works out to exactly one thing: `True` or `False`.

## 1. The order C# uses

C# always works an expression out in this order, top to bottom. Operators on the **same row** go **left to right**.

| Order | Operators | What they do |
|---|---|---|
| 1 | `( )` | Parentheses first, innermost pair first |
| 2 | `!` `(double)` `(int)` | Flip a bool, or cast a number. These apply only to the **one** value right after them. |
| 3 | `*` `/` `%` | Multiply, divide, remainder |
| 4 | `+` `-` | Add, subtract |
| 5 | `<` `>` `<=` `>=` | Compare sizes. The answer is a bool. |
| 6 | `==` `!=` | Equal, not equal. The answer is a bool. |
| 7 | `&&` | And |
| 8 | `\|\|` | Or |

Two parts of this table catch the most people:

- `&&` **comes before** `||`, the same way `*` comes before `+`. So `a || b && c` means `a || (b && c)`.
- `<` `>` `<=` `>=` **come before** `==` `!=`. So `a < b == c < b` means `(a < b) == (c < b)`.

## 2. And, or, not

| `x` | `y` | `x && y` | `x \|\| y` |
|---|---|---|---|
| true | true | **true** | **true** |
| true | false | false | **true** |
| false | true | false | **true** |
| false | false | false | false |

- `&&` (and) is true only when **both** sides are true. One false side makes it false.
- `||` (or) is true when **at least one** side is true. It's only false when both sides are false.
- `!` (not) flips one value: `!true` is `false`, `!false` is `true`.
- `!` only flips what's right after it. `!x == y` flips `x`, then compares. To flip a whole group, put the group in parentheses: `!(x == y)`.

## 3. Number rules that change the answer

| Rule | Example | Result |
|---|---|---|
| An `int` divided by an `int` chops off the decimals. It never rounds. | `17 / 7` | `2` |
| `%` gives the remainder. | `17 % 7` | `3` |
| An even number `% 2` is 0. An odd number `% 2` is 1. | `7 % 2` | `1` |
| A cast applies to the one value right after it, **before** the division. | `(double)12 / 5` | `2.4` |
| Without the cast, it's still int division. | `12 / 5` | `2` |
| Subtraction goes left to right. | `5 - 12 - 7` | `-14` (not `0`) |

## 4. Comparison rules

| Rule | Example | Result |
|---|---|---|
| `<` and `>` do **not** include the number itself. | `12 > 12` | `false` |
| `<=` and `>=` **do** include it. | `12 >= 12` | `true` |
| `==` asks a question; `=` assigns. An `if` or a bool needs `==`. | `a == 5` | `true` or `false` |
| `==` and `!=` can compare two bools. | `true == true` | `true` |
| | `true != false` | `true` |
| A range needs two comparisons joined with `&&`. `13 <= age <= 19` won't build. | `age >= 13 && age <= 19` | |

## 5. A method that works every time

1. **Replace** every variable with its value.
2. **Work the parentheses**, innermost first. Treat each one as a small problem of its own.
3. **Do the `!`s and casts.**
4. **Do the arithmetic**, `*` `/` `%` before `+` `-`. Watch for int division.
5. **Turn every comparison into `true` or `false`**: first `<` `>` `<=` `>=`, then `==` `!=`.
6. **Do every `&&`.**
7. **Do every `||`.**
8. **Write one step per line.** If you get it wrong, you can find the exact line where it went off track.

## 6. Worked example

With `int x = 4;`, `int y = 9;` and `bool done = false;`:

```
!done && y / x * x != y || x % 2 == 1

Step 1, replace:      !false && 9 / 4 * 4 != 9 || 4 % 2 == 1
Step 3, the !:        true && 9 / 4 * 4 != 9 || 4 % 2 == 1
Step 4, arithmetic:   9 / 4 = 2 (int division), 2 * 4 = 8; 4 % 2 = 0
                      true && 8 != 9 || 0 == 1
Step 5, comparisons:  true && true || false
Step 6, the &&:       true || false
Step 7, the ||:       True
```

# 7. Traps to watch for

- **Stopping early on `||`.** A true value only wins an `||` if it's a whole side of that `||`. In `false || true && false`, the `true` belongs to the `&&`, which is false. Answer: false.
- **Working left to right across `&&` and `||`.** Always do every `&&` before any `||`.
- **Using 2.4 when it should be 2.** If both numbers are ints, the division chops.
- **Forgetting the edge value.** `>` and `<` leave the number itself out.
- **Flipping too much or too little with `!`.** Without parentheses, `!` flips one value. With parentheses, it flips the whole group.
- **Thinking a bool can't be compared.** `(a < b) == (c < b)` is allowed. It asks whether both answers are the same.

# 8. When you write your own code

Parentheses never hurt. Even when C# doesn't need them, they make your intent obvious to the next person reading it, including you.

```
bool canEnter = isMember || (hasTicket && age >= 18);
```